

DAKAR MAD MAX

Informe de Desarrollo de Software

1. Introducción

Dakar Mad Max es un videojuego de escritorio desarrollado en C++ con Qt Creator como proyecto individual para la asignatura Informática II. El juego combina dos niveles de distinta naturaleza: una carrera de vista cenital top-down en el Nivel 1 y un runner lateral en el Nivel 2, ambientados en un universo postapocalíptico inspirado en la saga Mad Max y la competencia del Dakar.

El público objetivo son jugadores casuales interesados en juegos de acción y velocidad, con control simple mediante teclado y una curva de aprendizaje baja pero una mecánica que recompensa la estrategia de posicionamiento.

2. Definición General y Objetivos

2.1 Descripción general

El juego consta de dos niveles encadenados. En el Nivel 1 el jugador compite en una carrera cenital de 2 minutos contra un rival controlado por IA, sobre un desierto con terrenos variables (carretera, fango, llantas en llamas). En el Nivel 2 el jugador enfrenta un runner lateral con agujeros que debe saltar al presionar la tecla correcta dentro de un tiempo límite.

2.2 Objetivos

- Implementar una jerarquía de clases con herencia propia (Terreno → Fango / Carretera / Llanta; Vehículo → VehículoIA).
- Desarrollar físicas de movimiento diferenciadas: multiplicadores de velocidad por terreno, derrape sinusoidal en fango, bloqueo por llanta, movimiento senoidal de obstáculos.
- Implementar un agente inteligente que perciba el entorno, decida una acción y la ejecute cada frame.
- Integrar audio, animación de sprites y una interfaz de selección de vehículo funcional.
- Aplicar contenedores STL (std::vector, std::set) para la gestión de entidades dinámicas.

3. Metodología y Planificación

El desarrollo siguió una metodología incremental e iterativa. Cada funcionalidad fue implementada, probada manualmente y refinada antes de pasar a la siguiente, sin un sprint formal pero sí con entregas parciales alineadas a los Momentos del curso.

Fase	Actividades principales	Resultado
Momento I	Definición del concepto, descripción narrativa del juego, bocetos de niveles.	Documento de idea central.
Momento II	Diseño del diagrama de clases de la capa lógica, descripción de físicas y vistas, sprites.	Diagrama UML y documento de diseño.
Desarrollo base	Implementación de MainWindow, Escenario, Vehiculo con WASD y sprites.	Juego navegable sin lógica de terreno.
Físicas Nivel 1	Clases Terreno, Fango, Carretera, Llanta con multiplicadores y derrape.	Terrenos funcionales con efectos diferenciados.
Agente IA	Clase VehiculoIA con percepción, decisión y acción.	Rival que esquiva y busca carretera.
Nivel 2	Vista lateral, agujeros, mecánica de salto con tecla y meta.	Segundo nivel jugable.
Pulido	Audio, menú, detección de ganador, contador de tiempo.	Producto final integrado.

3. Diseño y Arquitectura del Sistema

3.1 Estructura general

El sistema se organiza en tres capas lógicas: presentación (Qt Widgets, QGraphicsScene), lógica de juego (clases de vehículos, terrenos y IA) y gestión (MainWindow como orquestador del game loop).

3.2 Jerarquías de herencia

Clase base	Subclases
Vehiculo (QGraphicsPixmapItem)	VehiculoIA — añade percepción y toma de decisiones automática.
Terreno (QGraphicsPixmapItem)	Fango — aplica multiplicador 0.5 y derrape sinusoidal.

	Carretera — aplica multiplicador 1.1, permite avance rápido.
	Llanta — movimiento senoidal, multiplicador -1 (bloqueo).

3.3 Componentes principales

Clase / Componente	Responsabilidad
MainWindow	Orquestador central. Maneja el game loop (QTimer 16ms), generación de terrenos, detección de colisiones y transición entre niveles.
Vehiculo	Movimiento con teclado, físicas de fango (derrape sinusoidal), bloqueo por llanta, límites de pantalla.
VehiculoIA	Hereda de Vehiculo. Escanea terrenos en zona de visión, decide dirección y aplica el mismo sistema de físicas que el jugador.
Terreno	Clase abstracta con actualizar() y getMultiplicador() como métodos virtuales puros.
Fango / Carretera / Llanta	Subclases concretas de Terreno. Cada una define su pixmap, multiplicador y comportamiento de movimiento.
Escenario	Dos pares de QGraphicsPixmapItem para el fondo y los bordes laterales con scroll infinito.
SeleccionVehiculo	Pantalla de menú con tres botones que emiten señal vehiculoSeleccionado(TipoVehiculo).
Nivel2	Widget independiente con su propio QGraphicsScene, game loop y mecánica de agujeros + salto.

3.4 Contenedores STL utilizados

Contenedor	Uso
std::vector<Terreno*>	Lista de todos los terrenos activos en escena (fango, carretera, llantas). Se reconstruye cada frame eliminando los que salen de pantalla.
std::set<int>	Teclas presionadas en Vehiculo. Permite detectar múltiples teclas simultáneas sin duplicados.
QVector<QPixmap>	Frames de animación del spritesheet de la llanta y del vehículo en el Nivel 2.

3.5 Dependencias externas

- Qt 6.10 — módulos: Widgets, Core, Gui, Multimedia.
- Compilador MinGW 64-bit incluido con Qt Creator.

4. Desarrollo e Implementación

4.1 Game loop

El game loop principal se ejecuta mediante un QTimer con intervalo de 16 ms (~60 FPS). Cada tick llama a: desplazar el escenario, actualizar el jugador, actualizar la IA, generar terrenos según contadores independientes, mover todos los terrenos, detectar colisiones mediante intersección de QRectF en coordenadas de escena, y depurar el vector de terrenos eliminando los que salen de pantalla.

4.2 Sistema de físicas

La velocidad de avance del vehículo sigue la fórmula:

$$v_y = -\text{speed} \times \text{multiplicadorTerreno}$$

Si el multiplicador es 0 (sin terreno), el vehículo no avanza. Si es positivo (carretera = 1.1, fango = 0.5), avanza proporcionalmente. Si es -1 (llanta), el vehículo retrocede y queda bloqueado por 60 frames.

El derrape en fango usa una función sinusoidal acumulada: a mayor tiempo presionando la tecla lateral, mayor es la fuerza del derrape (hasta un máximo de 1.5 s de acumulación). Las componentes son:

$$v_x = \text{dir} \times \text{fuerza} \times \sin(\varphi) \quad \text{— movimiento lateral en curva}$$

$$v_y = \text{fuerza} \times (1 - \cos(\varphi)) \times 0.6 \quad \text{— retroceso proporcional}$$

4.3 Agente inteligente

La IA de VehiculolAI opera en tres pasos cada frame:

- Percepción: escanea todos los terrenos activos dentro de un rectángulo de visión de 200px hacia adelante y 60px a cada lado. Clasifica cada terreno como peligro (fango/llanta) o carretera.
- Decisión: si está sobre peligro, huye al lado con más espacio respecto al centro de la pista. Si ve peligro adelante, lo esquiva. Si ve carretera, se dirige a su centro.
- Acción: calcula v_x y v_y con las mismas fórmulas del jugador y llama a `setPos()`, respetando los mismos límites de pantalla.

4.4 Detección de colisiones

Se utiliza intersección de QRectF en coordenadas de escena en lugar de `collidesWithItem()` de Qt, ya que este último resultó inestable con ítems dinámicos. Los `boundingRect` de cada ítem se transforman a coordenadas globales con `mapToScene()` antes de comparar.

5. Procedimientos de Prueba

Las pruebas se realizaron de forma manual e incremental durante el desarrollo, validando cada componente antes de integrarlo al sistema completo.

Caso de prueba	Procedimiento	Resultado esperado / obtenido
Carga de sprites	Ejecutar el juego y seleccionar cada vehículo.	Sprite correcto con transparencia y orientación adecuada. ✓
Límites de pantalla	Mover el vehículo contra los bordes.	El vehículo se detiene al llegar al límite definido por tipo. ✓
Efecto del fango	Entrar en un charco de fango y presionar A/D.	Derrape sinusoidal visible; mayor efecto a mayor tiempo presionado. ✓
Efecto de la llanta	Hacer que la llanta toque al vehículo.	Vehículo bloqueado y retrocede 4px/frame por 60 frames. ✓
Generación de terrenos	Esperar 200 frames desde el inicio.	Aparecen franjas de carretera y fango en posiciones aleatorias. ✓
Comportamiento IA	Observar al rival durante 2 minutos.	La IA esquiva charcos y se dirige a carreteras visibles. ✓ (básico)
Contador de tiempo	Iniciar el nivel y esperar.	Contador decrece correctamente hasta 0:00. ✓
Línea de meta	Dejar el tiempo agotarse.	Línea blanca aparece y baja a la velocidad del mundo. ✓
Detección de ganador	Posicionar el jugador en la trayectoria de la meta.	Mensaje correcto de victoria o derrota según quién cruce. ✓
Transición Nivel 1→2	Ganar el Nivel 1.	La ventana cambia a vista lateral del Nivel 2. ✓

6. Resultados

6.1 Funcionamiento logrado

El juego es completamente jugable en sus dos niveles. El Nivel 1 presenta una carrera con físicas diferenciadas por terreno, un rival con IA funcional y una mecánica de carrera con línea de meta. El Nivel 2 ofrece una experiencia distinta con saltos y respuesta rápida a estímulos visuales. La arquitectura de clases respeta la jerarquía de herencia requerida y hace uso extensivo de polimorfismo para el manejo uniforme de los terrenos.

6.2 Limitaciones actuales

- La IA del rival es reactiva y no predictiva

- No existe un sistema de puntuación persistente ni registro de mejores tiempos.
- La velocidad del mundo es constante; no hay dificultad progresiva en el Nivel 1.
- Las colisiones entre el jugador y el rival no están implementadas en el Nivel 1.

6.3 Dificultades enfrentadas

- La transparencia PNG no se mostraba correctamente usando QGraphicsPixmapItem; se resolvió migrando el renderizado a QPainter y posteriormente identificando que el problema era la configuración del archivo .qrc (prefijo duplicado en la ruta).
- El crash al generar terrenos fue causado por el uso de QGraphicsRectItem con parent=this dentro de un QGraphicsPixmapItem, lo que generaba jerarquías inválidas de items. Se resolvió dibujando directamente sobre el pixmap de la clase base.
- El agente IA no se movía inicialmente porque los métodos helper de Vehiculo estaban declarados como private en lugar de protected, impidiendo el acceso desde VehiculoIA.
- Las colisiones con los agujeros en el nivel 2 no se detectaban bien debido al tamaño de estos mismos y esto se solucionó reduciendo el tamaño.

6.4 Mejoras planeadas

- Implementar colisiones físicas entre el jugador y el rival (bloqueo lateral mutuo).
- Añadir un tercer tipo de terreno: tormenta de arena con efecto de viento lateral.
- Dificultad progresiva en el Nivel 2: velocidad creciente con el tiempo.
- Pantalla de resultados con estadísticas de la carrera.

6. Conclusiones

El desarrollo de Dakar Mad Max permitió consolidar de forma práctica los conceptos fundamentales de la programación orientada a objetos: la jerarquía Vehiculo → VehiculoIA y Terreno → {Fango, Carretera, Llanta} demostró el valor real del polimorfismo, permitiendo que el game loop trate todos los terrenos de forma uniforme con una sola llamada a actualizar() sin importar su tipo concreto.

El proceso de depuración fue especialmente formativo: la mayoría de los errores críticos (crasheos, IA sin movimiento, transparencia rota) tuvieron causas sutiles relacionadas con el manejo de memoria, los niveles de acceso en C++ y las particularidades del sistema de recursos de Qt. Enfrentar y resolver estos problemas de forma independiente desarrolla habilidades de diagnóstico que no se adquieren en ejercicios teóricos.

Finalmente, el proyecto evidenció que el desarrollo de software en solitario exige una planificación cuidadosa del alcance: es preferible implementar pocas funcionalidades bien hechas y robustas que muchas funcionalidades incompletas o inestables.

7. Referencias

- Qt Documentation (Qt 6.10). Qt Company. <https://doc.qt.io>
- Sprites generados con asistencia de IA (ChatGPT Image Generation, 2026).
- Música de fondo: Banda sonora mad max fury road

8. Anexos

Anexo A — Estructura de archivos del proyecto

Archivo	Descripción
main.cpp	Punto de entrada. Crea QApplication y MainWindow.
mainwindow.h / .cpp	Orquestador: game loop, generación de terrenos, colisiones, transición de niveles.
vehiculo.h / .cpp	Clase base del vehículo jugador con físicas completas.
vehiculoia.h / .cpp	Subclase de Vehiculo con percepción, decisión y acción automática.
terreno.h / .cpp	Clase abstracta base de todos los objetos de terreno.
fango.h / .cpp	Charco de fango: multiplicador 0.5, sprite fango.png.
carretera.h / .cpp	Tramo de carretera: multiplicador 1.1, sprite carretera.png.
llanta.h / .cpp	Llanta animada: movimiento senoidal, multiplicador -1 (bloqueo).
escenario.h / .cpp	Fondo y bordes laterales con scroll infinito.
seleccionvehiculo.h / .cpp	Pantalla de menú de selección con sprites e información por vehículo.
nivel2.h / .cpp	Nivel 2 completo: vista lateral, agujeros, salto y meta.
tiposvehiculo.h	Enum class TipoVehiculo: Moto, CarroDakar, Camion.
recursos.qrc	Recurso Qt con todos los sprites, fondos y audio.

Anexo B — Fragmento de código: derrape sinusoidal en fango

```
// En vehiculo.cpp - calcularMovimientoEnFango() derrapePhi += 0.08f; vx =
derrapeDirX * derrapeFuerza * std::sin(derrapePhi); vy = derrapeFuerza * (1.0f -
std::cos(derrapePhi)) * 0.6f; // Mayor tiempo presionando A/D → mayor derrapeFuerza
→ curva más exagerada
```

Anexo C — Fragmento de código: percepción del agente IA

```
// En vehiculoia.cpp - escanearTerrenos() QRectF vision(miRect.left() - 30,
miRect.top() - 200.0f, miRect.width() + 60, 200.0f); for (Terreno *t
: terrenos) { QRectF rT = t->mapToScene(t->boundingRect()).boundingRect();
if (vision.intersects(rT) && dynamic_cast<Fango*>(t)) info.peligroAdelante
= true; }
```